



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计期末实验报告

MPI 编程

颜国庄

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

目录

一、 并行算法设计	1
(一) 并行算法思路	1
(二) 并行算法具体实现	2
(三) 并行算法的正确性验证	6
二、 性能对比	7
(一) 不同进程数对比	7
(二) SIMD 与 MPI+SIMD 性能对比	8
三、 profiling 分析	9
(一) 任务池负载统计分析	9
(二) 通信、内存申请与数据拷贝开销分析	11
(三) Profiling 小结	13
四、 代码链接	13

一、 并行算法设计

(一) 并行算法思路

代码中使用结构体 `Block` 表示一个尚未完全收敛的子矩阵区间。

```
1 struct Block
2 {
3     int l;
4     int r;
5 };
```

其中, $[l, r]$ 表示一个活动子矩阵。如果 $l = r$, 说明该子问题已经退化为 1×1 , 不需要继续进行 bulge chasing; 如果 $r > l$, 则说明该活动块仍然需要继续迭代。

根据手册设计主进程维护一个子矩阵任务池, 工作进程从任务池中取出一个非 1×1 子矩阵后, 不再只执行一轮 bulge chasing, 而是在该子矩阵内部连续执行 `one_block_step`, 直到该子矩阵可以再次分裂。随后, 工作进程将分裂得到的新子矩阵返回给主进程, 再由主进程将新的非平凡子块重新放回任务池。

因此, 改进后的迭代逻辑可以概括为:

1. 从任务池中取出一个非 1×1 子矩阵;
2. 在该子矩阵内部反复执行 bulge chasing;
3. 直到该子矩阵可以再次分裂;
4. 将分裂后的非平凡子矩阵重新放回任务池。

MPI 版本与 pthread 版本的关键区别在于内存模型不同。pthread 中多个线程共享同一进程的地址空间, 可以直接访问和修改同一份 U 、 B 、 V 矩阵; 而 MPI 中每个进程都有独立地址空间, 工作进程对本地矩阵副本的修改不会自动同步到主进程。因此, 选择采用主从式任务池结构。rank 0 作为主进程, 负责维护任务池、分发任务、同步矩阵状态、接收结果并合并矩阵更新; rank 1 到 rank $p - 1$ 作为工作进程, 负责接收子矩阵任务并执行局部迭代。

并行算法流程如下:

1. 进行 MPI 环境初始化, 获取 rank 和 size, 并控制非 rank 0 进程的标准输出;
2. 以 rank 0 的 U 、 B 、 V 为基准, 将初始矩阵状态广播给所有进程;
3. rank 0 对上二对角矩阵进行预处理, 并将所有非 1×1 子矩阵加入任务池;
4. rank 0 从任务池中取出一个非平凡子矩阵, 发送给空闲 worker, 并同步当前最新的 U 、 B 、 V ;
5. worker 在收到的子矩阵内部连续执行 `one_block_step`, 直到该子矩阵可以再次分裂, 或者达到当前任务步数限制;
6. worker 回传局部矩阵 patch、执行步数以及分裂得到的新子矩阵集合;
7. rank 0 合并 worker 返回的 patch, 并将新产生的非 1×1 子矩阵重新放入任务池;
8. 当任务池为空或达到最大迭代步数后, rank 0 广播最终 U 、 B 、 V , 随后执行奇异值非负化和排序。

(二) 并行算法具体实现

MPI 初始化与输出控制

程序首先检测 MPI 环境是否已经初始化,如果尚未初始化,则由 `gkh.cpp` 自动调用 `MPI_Init`。同时代码通过 `atexit` 注册最终清理函数,在程序真正退出时再统一释放 MPI 环境。

```

1 static bool g_mpi_initialized_by_gkh = false;
2 static bool g_mpi_atexit_registered = false;
3 static bool g_stdout_redirected = false;
4
5 static void finalize_mpi_at_exit()
6 {
7     int mpi_initialized = 0;
8     int mpi_finalized = 0;
9
10    MPI_Initialized(&mpi_initialized);
11    MPI_Finalized(&mpi_finalized);
12
13    if (mpi_initialized && !mpi_finalized && g_mpi_initialized_by_gkh)
14    {
15        MPI_Finalize();
16    }
17 }
```

我们添加了输出控制逻辑: 获取当前进程编号后, 将非 rank 0 进程的标准输出 `stdout` 重定向到 `/dev/null`, 只保留 rank 0 的正常测试输出。

```

1 if (rank != 0 && !g_stdout_redirected)
2 {
3     std::fflush(stdout);
4     freopen("/dev/null", "w", stdout);
5     g_stdout_redirected = true;
6 }
```

这样处理后, 所有进程仍然会完整参与 MPI 通信和计算, 但最终结果只由 rank 0 输出。负载统计信息使用 `stderr` 输出, 因此不会受到标准输出重定向的影响。

初始子矩阵任务池的构造

进入 `gkh_svd_from_bidiagonal` 后, 程序首先以 rank 0 的矩阵为基准, 将初始 U 、 B 、 V 广播给所有进程, 保证每个 MPI 进程都拥有一致的初始状态。

```

1 broadcast_matrix_from_rank0(U);
2 broadcast_matrix_from_rank0(B);
3 broadcast_matrix_from_rank0(V);
```

随后, rank 0 对上二对角矩阵进行预处理, 包括清理数值噪声以及处理对角元接近 0 的特殊情况, 然后根据超对角线元素是否收敛划分初始活动块。

```

1 cleanup_bidiagonal(B, tol);
2 handle_diagonal_zeros(U, B, V, tol);
```

```

3
4 std::vector<Block> initial_blocks =
5 split_active_blocks(B, n, tol);

```

其中所有满足 $r > l$ 的活动块都会被加入子矩阵任务池；已经退化为 1×1 的块不再加入任务池，从而避免对已收敛子问题的重复处理。

```

1 std::vector<Block> task_pool;
2
3 for (const Block &blk : initial_blocks)
4 {
5     if (blk.r > blk.l)
6     {
7         task_pool.push_back(blk);
8     }
9 }

```

子矩阵内部的连续迭代

代码中设计了 `process_block_until_resplit` 函数。该函数接收一个活动块 $[l, r]$ ，在该区间内部连续调用 `one_block_step`。每执行一轮后，程序都会清理上二对角矩阵中的数值噪声，并在当前区间内重新检测是否产生新的分裂点。

```

1 while (steps_done < max_steps_for_task)
2 {
3     one_block_step(U, B, V, blk.l, blk.r);
4     ++steps_done;
5
6
7     cleanup_bidiagonal(B, tol);
8
9     std::vector<Block> after =
10         split_active_blocks_in_range(B, blk.l, blk.r, tol);
11
12     if (!same_single_block(after, blk))
13     {
14         return keep_nontrivial_blocks(after);
15     }
16
17
18 }

```

其中，`split_active_blocks_in_range` 只在当前子矩阵区间 $[l, r]$ 内检查分裂情况，而不是每次都重新扫描整个矩阵；`keep_nontrivial_blocks` 用于过滤掉已经收敛的 1×1 子块，只保留仍然需要继续迭代的非平凡子矩阵。如果在给定的步数预算内该子矩阵仍未分裂，则函数会把原子矩阵重新返回给主进程，由主进程再次放回任务池，避免单个任务长时间占用某一个 worker。

主从式任务池调度

在任务调度中, rank 0 作为 master, 专门负责维护任务池和合并结果; rank 1 到 rank $p-1$ 作为 worker, 负责实际的子矩阵计算。master 首先从任务池中取出若干非平凡子矩阵发送给空闲 worker。当某个 worker 完成任务后, 它会把局部矩阵更新和新分裂出的子矩阵返回给 master。master 合并该 worker 的计算结果后, 再把新产生的非 1×1 子矩阵放回任务池, 并继续向空闲 worker 分发任务。

该过程可以用伪代码表示如下:

Algorithm 1 改进迭代逻辑后的 MPI 主从式任务池调度算法

Input: 初始活动块集合, 矩阵 U, B, V , 最大迭代步数

Output: 完成 GKH 子矩阵池迭代

```

1: if rank = 0 then
2:   将所有非  $1 \times 1$  初始活动块加入任务池
3:   将全局执行步数置为 0
4:   向空闲 worker 分发任务
5:   while 仍有 worker 处于活动状态 do
6:     接收某个 worker 返回的 patch、执行步数和新子矩阵集合
7:     将 patch 合并到 master 的  $U, B, V$ 
8:     更新全局执行步数
9:     将新产生的非  $1 \times 1$  子矩阵放回任务池
10:    if 任务池非空且未达到最大迭代步数 then
11:      向该 worker 发送下一个任务
12:    else
13:      向该 worker 发送停止信号
14:    end if
15:  end while
16:  广播最终更新后的  $U, B, V$ 
17: else
18:  while true do
19:    接收 rank 0 发来的任务或停止信号
20:    if 收到停止信号 then
21:      退出 worker 循环
22:    else
23:      接收 master 同步的最新  $U, B, V$ 
24:      对该子矩阵连续执行 GKH 迭代直到再次分裂
25:      回传局部 patch、执行步数和新子矩阵集合
26:    end if
27:  end while
28:  接收 rank 0 广播的最终  $U, B, V$ 
29: end if

```

这种动态任务池方式比静态划分更适合活动块规模不一致的情况。因为不同子矩阵的长度和收敛速度不同, 计算量可能差异较大。动态调度允许先完成任务的 worker 继续领取新任务, 从而在一定程度上缓解负载不均衡问题。

MPI 消息 tag 的设计

为了避免不同类型的 MPI 消息相互混淆，代码为任务、停止信号、结果头信息、矩阵 patch、新子块以及完整矩阵同步分别设置了不同的 tag。主要 tag 在下表列出。这些 tag 分别用于任务发送、停止信号、结果头信息、局部矩阵 patch、新子块回传以及完整矩阵状态同步。

代码中对应的 tag 定义如下：

```
1 enum MpiTag
2 {
3     TAG_TASK = 100,
4     TAG_STOP_ROUND = 101,
5     TAG_RESULT_HEADER = 102,
6     TAG_RESULT_TIME = 103,
7     TAG_RESULT_U = 104,
8     TAG_RESULT_B = 105,
9     TAG_RESULT_V = 106,
10    TAG_RESULT_BLOCKS = 107,
11    TAG_STATE_U_DIMS = 108,
12    TAG_STATE_U_DATA = 109,
13    TAG_STATE_B_DIMS = 110,
14    TAG_STATE_B_DATA = 111,
15    TAG_STATE_V_DIMS = 112,
16    TAG_STATE_V_DATA = 113
17 };
```

矩阵状态同步与 patch 回传

worker 在接收任务后，需要先接收 master 发送的最新矩阵状态，确保本次计算基于 rank 0 已经合并过的 U 、 B 、 V 。这是因为 MPI 进程之间不共享内存，worker 本地矩阵副本可能落后于 master 的最新状态。

```
1 recv_full_state_from_master(U, B, V);
```

worker 完成子矩阵内部的连续迭代后，并不回传完整矩阵，而是只回传当前活动块影响到的局部数据。对活动块 $[l, r]$ ，回传内容包括 U 的第 l 到第 r 列、 B 的 $[l:r, l:r]$ 子块、 V 的第 l 到第 r 列，以及本次任务分裂得到的新子矩阵集合。

```
1 std::vector<double> u_patch = extract_cols(U, blk.l, blk.r);
2 std::vector<double> b_patch = extract_square_block(B, blk.l, blk.r);
3 std::vector<double> v_patch = extract_cols(V, blk.l, blk.r);
```

rank 0 收到 patch 后，将其合并回自己的矩阵，并将 worker 返回的新子矩阵重新加入任务池。

```
1 apply_cols(U, blk.l, blk.r, u_patch);
2 apply_square_block(B, blk.l, blk.r, b_patch);
3 apply_cols(V, blk.l, blk.r, v_patch);
4
5 for (const Block &b : new_blocks)
6 {
```

```

7  if (b.r > b.l)
8  {
9  task_pool.push_back(b);
10 }
11 }

```

在任务池全部处理结束后, rank 0 会广播最终的 U 、 B 、 V , 确保所有进程在函数返回前拥有一致的最终矩阵状态。

```

1 broadcast_matrix_from_rank0(U);
2 broadcast_matrix_from_rank0(B);
3 broadcast_matrix_from_rank0(V);

```

这种完整状态同步与局部 patch 回传相结合的方式可以保证计算结果的一致性, 但也会带来较大的通信和数据拷贝开销, 尤其在 1000×1000 矩阵规模下, 完整矩阵同步会成为影响 MPI 版本性能的重要因素。

计时与负载统计

为了分析 MPI 任务池的负载均衡情况, 代码记录了每个 rank 完成的任务数量、估计工作量以及局部计算时间。由于改进后的任务可能在一个子矩阵中连续执行多次 `one_block_step`, 因此估计工作量不再只按子矩阵长度计算, 而是进一步乘以该任务实际执行的步数。

```

1 local_tasks++;
2 local_work += block_work(blk) * std::max(1, steps);
3 local_compute_time += (t1 - t0);

```

其中, `local_tasks` 表示当前进程完成的子矩阵任务数量, `local_work` 表示估计工作量, `local_compute_time` 表示 worker 实际执行局部 GKH 计算的累计时间。程序最后使用 `MPI_Reduce` 汇总所有进程的统计结果。

```

1 MPI_Reduce(&local_tasks, &global_tasks, 1,
2 MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
3
4 MPI_Reduce(&local_work, &global_work, 1,
5 MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
6
7 MPI_Reduce(&local_compute_time, &global_compute_time, 1,
8 MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

```

在编译时加入宏 `-DSVD_PRINT_LOAD`, 程序会输出各进程的负载统计信息。

(三) 并行算法的正确性验证

输入 `mpicxx -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main_mpi -march=armv8-a` 进行编译。 `mpirun -np 4 ./main_mpi` 进行运行。得到结果如下:


```

=== 随机 1000x1000 ===
converged                      : yes
||A-U*S*V^T||_F                : 2.06312e-10
relative recon error           : 3.56932e-13
||U^T U-I||_F                  : 2.30887e-13
||V^T V-I||_F                  : 2.28851e-13
diagonal structure error       : 0
descending order error         : 0
nonnegative diagonal           : yes
time bidiagonalization(ms)     : 5315.34
time gkh iteration(ms)         : 76829.8
结果: PASS

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 5315.4
总GKH迭代耗时(ms): 76831.4
通过: 5 / 5
[s2413140@master_ubss1 svd]$ 

```

可以看到结果没有问题都是通过，但是时间似乎拉长了。

二、性能对比

(一) 不同进程数对比

性能测试阶段不打开负载统计宏，以避免额外输出对总耗时观察造成干扰。编译命令如下：

```

1  mpicxx -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main_mpi -
    march=armv8-a

```

在末尾通过对-DTEST_SEED 进行赋值来指定种子值。

运行时分别设置 1、2、4、8 个 MPI 进程，观察不同进程数下程序的总耗时变化：

```

1  mpirun -np 1 ./main_mpi > mpi_1.txt 2>&1
2  mpirun -np 2 ./main_mpi > mpi_2.txt 2>&1
3  mpirun -np 4 ./main_mpi > mpi_4.txt 2>&1
4  mpirun -np 8 ./main_mpi > mpi_8.txt 2>&1

```

表 1: 不同种子和 MPI 进程数下的 GKH 迭代耗时

种子值	1 进程	2 进程	4 进程	8 进程
20260408	34104.8ms	66129.4ms	65141.2ms	78280ms
20260409	32273.8ms	67290.4ms	68136.4ms	81078.9ms
20260410	34134.3ms	71240ms	57816.1ms	71673.1ms
20260411	32171.7ms	61218.1ms	65630.2ms	138444ms
20260412	43046.7ms	79511.9ms	159019ms	300030ms

从表中可以看出, 当前 MPI 版本并没有随着进程数增加而稳定加速。对于所有随机种子, 1 进程下的 GKH 迭代耗时均低于 2、4、8 进程, 这初步说明当前实现中的 MPI 通信、任务调度和数据同步开销较大, 抵消了局部并行计算带来的收益。此外, 不同随机种子下的耗时也存在明显差异。这可能是因为不同种子生成的矩阵具有不同的数据分布, 进而影响迭代的收敛速度。当矩阵较难分裂出多个可并行的非平凡子矩阵时, MPI 任务池的并行度会受到限制, 通信和同步开销占比上升。

进程数增加并没有带来稳定加速, 1 进程反而是当前实现下最快的运行方式; 2 进程和 4 进程偶尔能体现一定并行收益, 但不稳定; 8 进程由于通信、同步和负载不均衡问题, 性能退化最明显。

综上, 虽然当前 MPI 版本在正确性上能够完成测试, 但性能提升并不理想。主要瓶颈可能在 MPI 并行框架引入的完整矩阵同步、patch 回传、master 合并结果以及 worker 负载不均衡等方面, 还需通过后续 profiling 进行分析。

(二) SIMD 与 MPI+SIMD 性能对比

为了保证对比的公平性, 两种版本均采用 -O2 优化等级和 -march=armv8-a 编译参数, 并使用相同的测试样例和随机种子。SIMD 版本使用普通 g++ 编译, MPI+SIMD 版本使用 mpicxx 编译。

```

1 g++ -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main -march=
   armv8-a
2
3 mpicxx -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main_mpi -
   march=armv8-a

```

实验结果如下表所示, 其中 mpi+simd 的实验数据与上文一致

表 2: SIMD 与 MPI+SIMD 耗时

种子值	SIMD	MPI+SIMD 2 进程	MPI+SIMD 4 进程	MPI+SIMD 8 进程
20260408	35262.1ms	66129.4ms	65141.2ms	78280ms
20260409	45715.8ms	67290.4ms	68136.4ms	81078.9ms
20260410	52732.1ms	71240ms	57816.1ms	71673.1ms
20260411	59178.1ms	61218.1ms	65630.2ms	138444ms
20260412	57134.8ms	79511.9ms	159019ms	300030ms

从表中可以看出, 在当前实现和测试规模下, MPI+SIMD 版本并没有相对于 SIMD 获得稳

定加速。相比之下，MPI+SIMD 版本在 2、4、8 进程下的耗时普遍更高，可能原因是 MPI 主从式任务池引入的通信、同步和结果合并开销在当前实验中占据了较大比例。

具体来看，2 进程 MPI+SIMD 在所有随机种子下均慢于 SIMD。这可能是由于 2 进程结构中 rank 0 主要承担 master 调度职责，实际只有 1 个 worker 进行子矩阵计算，因此并行计算收益有限，但任务分发、矩阵状态同步和 patch 回传等 MPI 开销已经存在，导致整体耗时高于 SIMD。4 进程 MPI+SIMD 在部分种子下相比 2 进程有所改善，说明当 GKH 迭代过程中能够产生较多可并行的非平凡子矩阵时，增加 worker 数量可能带来一定收益。8 进程 MPI+SIMD 的性能退化最明显。这表明当进程数继续增加时，master 需要处理更多 worker 的通信、同步和结果合并，额外开销进一步放大。如果任务池中可并行的活动块数量不足，更多 worker 无法持续获得有效计算任务，反而会增加等待和调度开销。

综上，SIMD 主要优化单进程内部的行列旋转计算，而 MPI+SIMD 虽然保留了 SIMD 优化，但额外引入了 MPI 任务池调度、完整矩阵同步、局部 patch 回传和 master 合并结果等开销。在当前实现中，MPI 带来的并行收益不足以抵消这些额外开销，因此 MPI+SIMD 整体表现不如 SIMD。该结果并不说明 SIMD 优化失效，而是说明当前 MPI 并行化方式的通信和同步开销较大，限制了 SIMD 与 MPI 结合后的整体性能提升。

三、 profiling 分析

(一) 任务池负载统计分析

在 gkh.cpp 的 MPI 任务池内部加入负载统计，记录每个 rank 完成的任务数量、估计工作量和局部计算时间。编译时通过 -DSVD_PRINT_LOAD 宏打开 profiling 输出，编译和运行命令如下：

```
1 mpicxx -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o  
   main_mpi_profile -march=armv8-a -DSVD_PRINT_LOAD  
2  
3 mpirun -np 4 ./main_mpi_profile > profile_np4.txt 2>&1  
4 mpirun -np 8 ./main_mpi_profile > profile_np8.txt 2>&1
```

为了观察任务池调度效果，代码中记录了每个 rank 的任务数量、估计工作量和局部计算时间。其中，tasks 表示该 rank 完成的子矩阵任务数量，work 表示根据子矩阵规模估计得到的工作量，compute_time 表示 worker 在局部子矩阵 bulge chasing 计算中消耗的累计时间。由于不同子矩阵的长度和收敛速度不同，仅使用 tasks 不能完全反映真实负载，因此需要结合 work 和 compute_time 共同分析。

4 进程下的负载统计结果：

```

[s2413140@master_ubss1 svd]$ mpicxx -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main_mpi_profile -march=armv8-a -DSVD_PRINT_LOAD
[s2413140@master_ubss1 svd]$ mpirun -np 4 ./main_mpi_profile > profile_np4.txt 2>&1
[s2413140@master_ubss1 svd]$ grep "mpi taskpool" profile_np4.txt
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 1: tasks=4, work=154, compute_time=0.000019 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh total] tasks=4, work=154, sum_compute_time=0.000019 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 1: tasks=7, work=548, compute_time=0.000019 s
[mpi taskpool gkh total] tasks=7, work=548, sum_compute_time=0.000019 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 1: tasks=7, work=506, compute_time=0.000018 s
[mpi taskpool gkh total] tasks=7, work=506, sum_compute_time=0.000018 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 1: tasks=7, work=608, compute_time=0.000023 s
[mpi taskpool gkh total] tasks=7, work=608, sum_compute_time=0.000023 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh total] tasks=994, work=538199309, sum_compute_time=86.654790 s
[mpi taskpool gkh load] rank 1: tasks=994, work=538199309, compute_time=86.654790 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s

```

从图中可以看出,当前实现存在负载不均衡现象。由于 rank 0 作为 master 主要负责任务池维护、任务分发、矩阵状态同步、局部 patch 接收和结果合并,因此其 tasks、work 和 compute_time 均为 0 是正常现象,并不表示 rank 0 没有开销,而是说明其开销主要体现在调度和通信过程中。

从大规模测试对应的统计结果看,rank 1 完成了主要计算任务,而 rank 2 和 rank 3 的 tasks、work 和 compute_time 均为 0。这说明在该次 4 进程运行中,实际参与子矩阵 bulge chasing 计算的主要只有 rank 1,其他 worker 基本处于空闲或等待状态,动态任务池没有形成理想的多 worker 并行计算。

结合前面的性能测试结果可以进一步解释 MPI+SIMD 未能稳定加速的原因。虽然程序启动了 4 个 MPI 进程,但由于迭代过程中可并行的非平凡子矩阵数量受矩阵分裂情况限制,任务池中并不总是存在足够多的独立任务可分配给多个 worker。同时, master 在任务分发时需要同步矩阵状态,worker 计算结束后还需要回传局部 patch 并由 master 合并结果,这些通信和同步开销会进一步增加总运行时间。该结果可以说明当前 MPI+SIMD 版本的主要瓶颈不在 SIMD 局部行列旋转计算本身,而在任务并行度不足、worker 负载不均衡以及 MPI 通信同步开销较大。

8 进程下的负载统计结果:

```

[s2413140@master_ubss1 svd]$ mpirun -np 8 ./main_mpi_profile > profile_np8.txt 2>&1
[s2413140@master_ubss1 svd]$ grep "mpi taskpool" profile_np8.txt
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh total] tasks=4, work=154, sum_compute_time=0.000019 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh total] tasks=7, work=548, sum_compute_time=0.000019 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh total] tasks=7, work=506, sum_compute_time=0.000019 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh total] tasks=7, work=608, sum_compute_time=0.000023 s
[mpi taskpool gkh load] rank 1: tasks=4, work=154, compute_time=0.000019 s
[mpi taskpool gkh load] rank 1: tasks=7, work=548, compute_time=0.000019 s
[mpi taskpool gkh load] rank 1: tasks=7, work=506, compute_time=0.000019 s
[mpi taskpool gkh load] rank 1: tasks=7, work=608, compute_time=0.000023 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 4: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 4: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 4: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 4: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 5: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 5: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 5: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 5: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 6: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 6: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 6: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 6: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 7: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 7: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 7: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 7: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 0: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh total] tasks=994, work=538199309, sum_compute_time=155.374496 s
[mpi taskpool gkh load] rank 1: tasks=994, work=538199309, compute_time=155.374496 s
[mpi taskpool gkh load] rank 2: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 3: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 4: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 5: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 6: tasks=0, work=0, compute_time=0.000000 s
[mpi taskpool gkh load] rank 7: tasks=0, work=0, compute_time=0.000000 s

```

从最后一组统计结果可以看出, rank 0 作为 master 进程, 主要负责任务池维护、任务分发、矩阵状态同步和结果合并, 因此其 `tasks`、`work` 和 `compute_time` 均为 0 是正常现象。然而, 在 rank 1 至 rank 7 这些 worker 中, 只有 rank 1 完成了实际计算任务, 其 `tasks` 为 994, `work` 为 538199309, 局部计算时间为 155.374496s; 其余 rank 2 至 rank 7 的任务数、工作量和局部计算时间均为 0。

这一结果说明, 在 8 进程运行时, 当前 MPI+SIMD 版本并没有形成理想的多 worker 并行计算。虽然程序启动了 8 个 MPI 进程, 但实际参与子矩阵 bulge chasing 计算的主要仍然只有 rank 1, 其余 worker 基本处于空闲或等待状态。因此, 增加进程数并没有提升任务并行度, 反而可能引入更多进程调度、通信同步和等待开销。

结合前面的性能测试结果可以看出, 8 进程下 MPI+SIMD 的性能退化并不是偶然现象。GKH 迭代过程中可并行的非平凡子矩阵数量受到矩阵分裂情况限制, 当任务池中缺少足够多的独立活动块时, 即使增加 worker 数量, 也无法让更多进程参与有效计算。同时, master 需要维护更多 worker 的通信和状态同步, 这会进一步放大 MPI 开销。因此, 该 profiling 结果表明, 当前实现的主要瓶颈在于任务并行度不足、worker 负载严重不均衡以及 MPI 通信同步开销较大。

(二) 通信、内存申请与数据拷贝开销分析

为了进一步分析 MPI+SIMD 版本中通信、内存申请与数据拷贝带来的额外开销, 我们在任务池统计代码中增加了对完整矩阵同步次数、完整矩阵同步数据量、patch 回传次数、patch 回传

数据量、临时数组申请量、通信时间以及拷贝时间的统计。编译时仍然通过 `-DSVD_PRINT_LOAD` 打开 profiling 输出，测试命令如下：

```
1 mpicxx -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o
   main_mpi_profile -march=armv8-a -DSVD_PRINT_LOAD
2
3 mpirun -np 4 ./main_mpi_profile > profile_np4.txt 2>&1
4 grep "overhead" profile_np4.txt
5
6 mpirun -np 8 ./main_mpi_profile > profile_np8.txt 2>&1
7 grep "overhead" profile_np8.txt
```

其中，`full_sync_count` 表示完整矩阵状态同步次数，`full_state` 表示完整同步 U 、 B 、 V 的总数据量，`patch_count` 表示 worker 回传 patch 的次数，`patch` 表示 patch 回传数据量，`temp_alloc` 表示 worker 为构造临时 patch 数组而产生的临时缓冲区申请量，`comm_time` 表示 MPI 通信累计时间，`copy_time` 表示 patch 提取与合并过程中的数据拷贝时间。

4 进程运行结果：

```
[s241314@master_ubss1 svd]$ mpicxx -O2 -std=c++17 main.cpp bidiagonalization.cpp gkh.cpp -o main_mpi_profile -march=armv8-a -DSVD_PRINT_LOAD
[s241314@master_ubss1 svd]$ mpirun -np 4 ./main_mpi_profile > profile_np4.txt 2>&1
[s241314@master_ubss1 svd]$ grep "overhead" profile_np4.txt
[mpi taskpool gkh overhead] full_sync_count=4, full_state=0.002 MB, patch_count=4, patch=0.001 MB, temp_alloc=0.003 MB, comm_time=0.000042 s, copy_time=0.000004 s
[mpi taskpool gkh overhead] full_sync_count=7, full_state=0.010 MB, patch_count=7, patch=0.006 MB, temp_alloc=0.012 MB, comm_time=0.000033 s, copy_time=0.000007 s
[mpi taskpool gkh overhead] full_sync_count=7, full_state=0.013 MB, patch_count=7, patch=0.006 MB, temp_alloc=0.013 MB, comm_time=0.000037 s, copy_time=0.000005 s
[mpi taskpool gkh overhead] full_sync_count=7, full_state=0.013 MB, patch_count=7, patch=0.006 MB, temp_alloc=0.013 MB, comm_time=0.000035 s, copy_time=0.000004 s
[mpi taskpool gkh overhead] full_sync_count=994, full_state=22750.854 MB, patch_count=994, patch=10166.806 MB, temp_alloc=20333.611 MB, comm_time=25.390732 s, copy_time=9.160438 s
[s241314@master_ubss1 svd]$
```

8 进程运行结果：

```
[mpi taskpool gkh overhead] full_sync_count=994, full_state=22750.854 MB, patch_count=994, patch=10166.806 MB, temp_alloc=20333.611 MB, comm_time=25.390732 s, copy_time=9.160438 s
[s241314@master_ubss1 svd]$ mpirun -np 8 ./main_mpi_profile > profile_np8.txt 2>&1
[s241314@master_ubss1 svd]$ grep "overhead" profile_np8.txt
[mpi taskpool gkh overhead] full_sync_count=4, full_state=0.002 MB, patch_count=4, patch=0.001 MB, temp_alloc=0.003 MB, comm_time=0.000038 s, copy_time=0.000003 s
[mpi taskpool gkh overhead] full_sync_count=7, full_state=0.010 MB, patch_count=7, patch=0.006 MB, temp_alloc=0.012 MB, comm_time=0.000125 s, copy_time=0.000008 s
[mpi taskpool gkh overhead] full_sync_count=7, full_state=0.013 MB, patch_count=7, patch=0.006 MB, temp_alloc=0.013 MB, comm_time=0.000083 s, copy_time=0.000006 s
[mpi taskpool gkh overhead] full_sync_count=7, full_state=0.013 MB, patch_count=7, patch=0.006 MB, temp_alloc=0.013 MB, comm_time=0.000101 s, copy_time=0.000004 s
[mpi taskpool gkh overhead] full_sync_count=994, full_state=22750.854 MB, patch_count=994, patch=10166.806 MB, temp_alloc=20333.611 MB, comm_time=24.362511 s, copy_time=8.331487 s
[s241314@master_ubss1 svd]$
```

将数据统计为表格形式：

表 3: MPI+SIMD 通信与临时内存申请开销统计

进程数	完整同步次数	完整同步量/MB	patch 量/MB	临时申请量/MB
4	994	22750.854	10166.806	20333.611
8	994	22750.854	10166.806	20333.611

表 4: MPI+SIMD 通信时间与数据拷贝时间统计

进程数	patch 回传次数	通信时间/s	拷贝时间/s
4	994	25.390732	9.160438
8	994	24.362511	8.331487

从图中看出，当前 MPI+SIMD 实现中，worker 每处理一次子矩阵任务，通常都需要与 master 之间进行一次较重的数据交互。完整矩阵同步总量达到 22750.854MB；patch 回传总量达到 10166.806MB；同时，worker 构造 `u_patch`、`b_patch` 和 `v_patch` 等临时数组造成的临时

申请量达到 20333.611MB。这些数据说明, MPI+SIMD 版本中的额外开销并不是少量控制消息造成的, 而是由大规模矩阵状态同步、局部 patch 回传以及临时数组构造共同导致的。

进一步观察通信与拷贝时间, 虽然 8 进程下通信和拷贝时间略低于 4 进程, 但结合前面的负载统计可知, 8 进程并没有让更多 worker 参与有效计算, 实际计算仍主要集中在 rank 1。因此, 这些通信与拷贝开销并没有换来足够的并行计算收益, 反而成为影响整体性能的重要因素。

从代码逻辑看, 完整矩阵同步主要发生在 master 向 worker 下发任务前, 用于保证 worker 使用的是 rank 0 已经合并过的最新 U 、 B 、 V 状态。由于 MPI 进程之间不共享内存, worker 不能直接访问 master 中的矩阵, 因此必须通过显式通信获得最新数据。对于大规模矩阵, 这一步会产生大量通信数据。worker 完成子矩阵内部的 bulge chasing 后, 还需要提取当前活动块影响到的局部矩阵数据, 构造 u_patch 、 b_patch 和 v_patch , 再回传给 master。master 接收 patch 后, 还需要将这些局部数据重新写回自己的 U 、 B 、 V 。因此, 一个任务至少包含完整状态同步、worker 端 patch 构造、patch 回传以及 master 端 patch 合并等多个开销环节。

(三) Profiling 小结

综上, 我们代码没有实现加速, 但探究其原因还是可知一二。

首先, 4 进程和 8 进程下实际参与子矩阵计算的主要都是 rank 1, 其他 worker 的 `tasks`、`work` 和 `compute_time` 多数为 0。这说明虽然程序启动了多个 MPI 进程, 但 GKH 迭代过程中可并行的非平凡子矩阵数量有限, 任务池中并不总是存在足够多的独立任务。因此, 增加进程数并没有使多个 worker 同时承担有效计算, 反而使部分进程处于空闲或等待状态。

其次, 在 1000×1000 大规模测试中, 4 进程和 8 进程下均发生了 994 次完整矩阵同步和 994 次 patch 回传。完整矩阵同步总量达到 22750.854MB, patch 回传数据量达到 10166.806MB, 临时 patch 数组申请量达到 20333.611MB。这表明当前 MPI+SIMD 版本的额外开销主要来自大规模矩阵状态同步、局部 patch 构造与回传, 以及 master 端 patch 合并, 而不是少量控制消息本身。

然后, 4 进程下通信时间为 25.390732s, 拷贝时间为 9.160438s; 8 进程下通信时间为 24.362511s, 拷贝时间为 8.331487s。这些时间虽然不是程序总耗时的全部, 但已经说明通信和数据拷贝在整体运行中占据了不可忽略的比例。由于 worker 负载严重不均衡, 这些通信和拷贝开销没有换来足够的并行计算收益, 因此最终表现为 MPI+SIMD 相比 SIMD 有稳定加速, 甚至在部分种子和 8 进程下出现明显性能退化。

最后我认为可能也有一些关系的点是服务器的稳定性, 我在中午一点午休时间编译运行不同种子得到的数据较为稳定, 但在下午两点运行种子 20240412 时迭代时间变得高了很多。

profiling 结果说明: 当前 MPI+SIMD 版本的性能瓶颈主要来自三个方面。第一, GKH 迭代本身的可并行子矩阵数量受分裂情况限制, 导致任务池并行度不足; 第二, worker 之间负载分配不均, 实际计算集中在少数进程上; 第三, 完整矩阵同步、patch 回传、临时数组申请和数据拷贝带来了较大的 MPI 额外开销。综合这些因素, 当前实现虽然能够保证计算正确性, 但并未获得理想加速效果。

四、 代码链接

[点击访问 GitHub 主页](#)